

Informatica 2.2 / Computational Intelligence

Multi-objective optimization

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026



Università
Ca' Foscari
Venezia

Multi-objective Optimization (MOO)

So far, we discussed problems in which a **single fitness function** needed to be optimized

- E.g., fitting error, free energy, distance to be travelled...
- This is **seldom the case** in real world scenarios

Often, optimization problems are characterized by **multiple conflicting goals**

- Maximize the effect of a drug while minimizing the production costs
- Minimize time-to-destination while also minimizing the consumption of fuel
- Maximize coverage of wireless sensors, minimize energy consumption and costs
- Maximize the effectiveness of a drug while minimizing its toxicity
- Best wedding venue while minimizing the costs and maximizing bride's happiness...

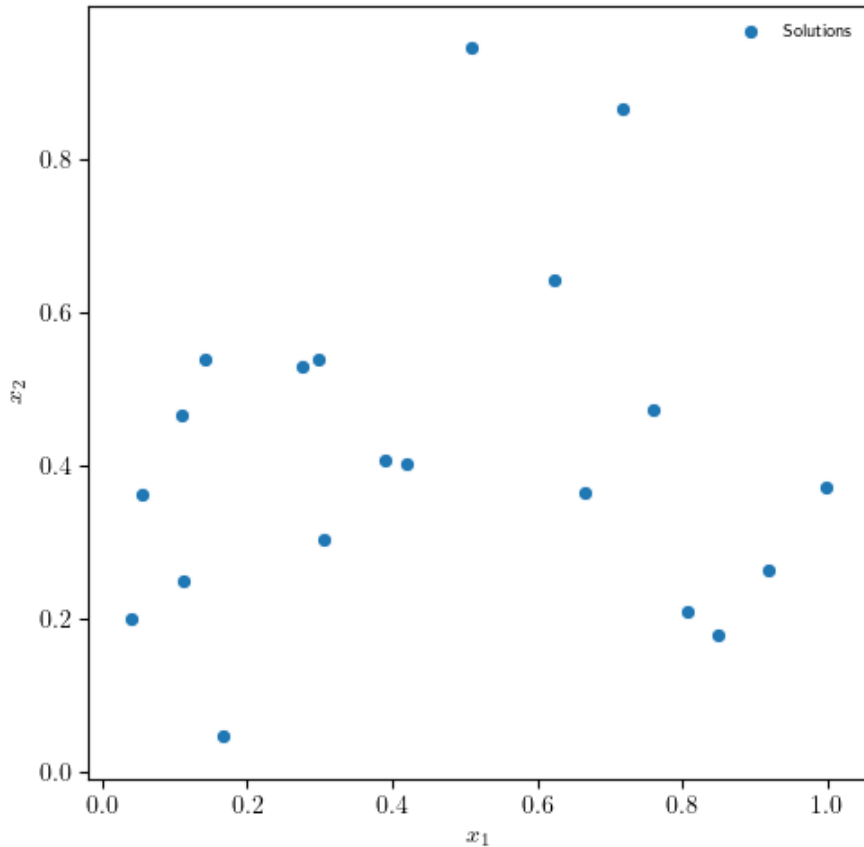


An abstract view of MOO

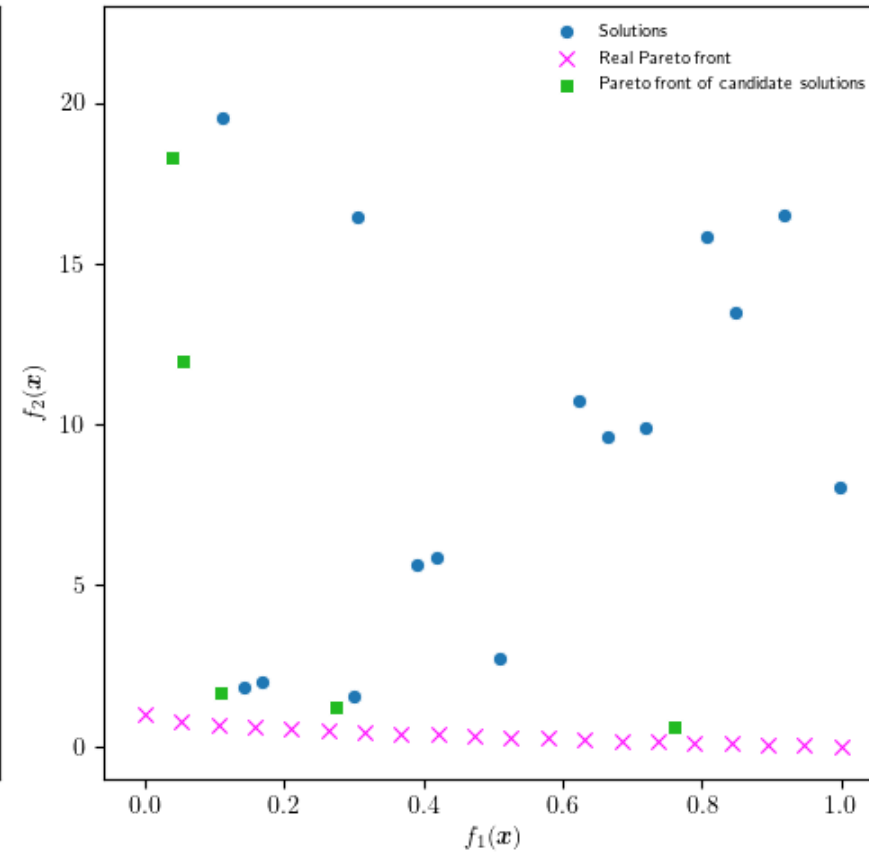
- We have **multiple fitness functions** $f_1, f_2, \dots, f_k$ to be simultaneously optimized
- They can be **conflicting**: if you improve f_1 , it might affect the other $k - 1$ values
- The concept of **fitness landscape** no longer makes sense

From solution space to objectives space

Solution space



Objectives space



ZDT-4 benchmark function

$$f_1(\vec{x}) = x_1$$

$$f_2(\vec{x}) = g(\vec{x}) [1 - (x_1/g(\vec{x}))^2]$$

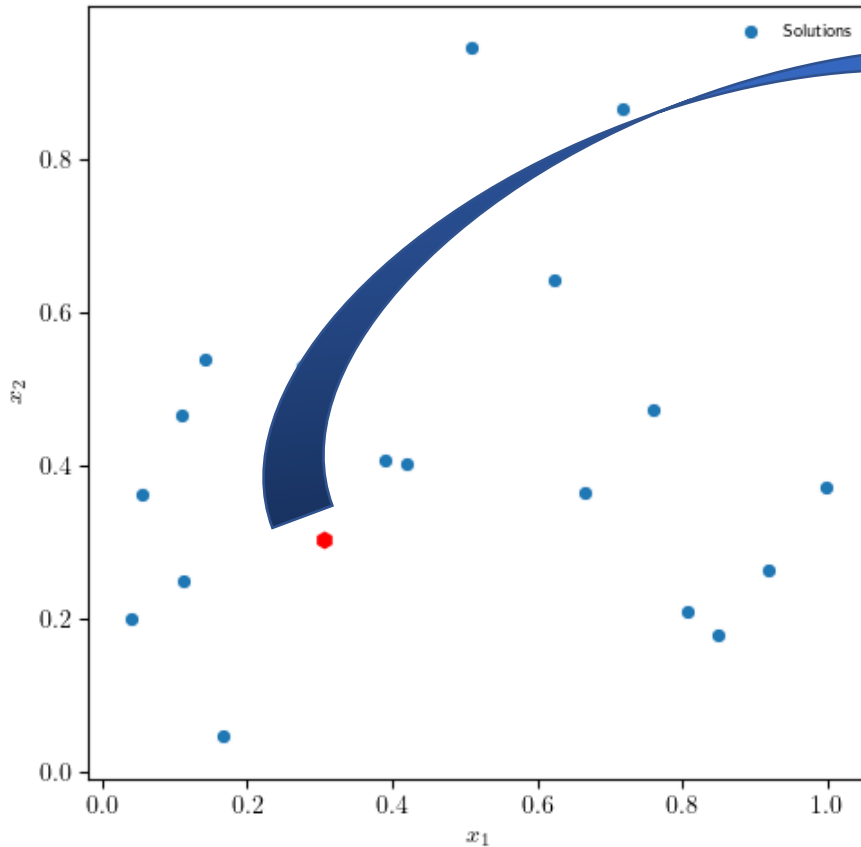
$$g(\vec{x}) = 1 + 10(n - 1) + \sum_{i=2}^n (x_i^2 - 10\cos(4\pi x_i))$$

$$0 \leq x_1 \leq 1$$

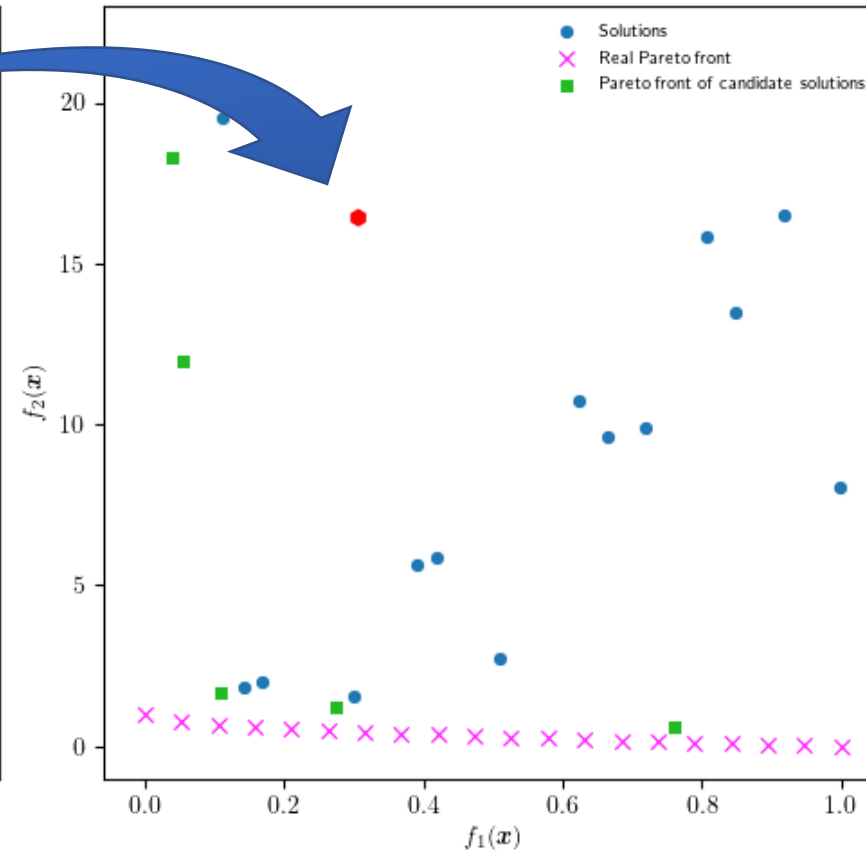
$$-5 \leq x_i \leq 5, i = 1, \dots, n$$

From solution space to objectives space

Solution space



Objectives space



ZDT-4 benchmark function

$$f_1(\vec{x}) = x_1$$

$$f_2(\vec{x}) = g(\vec{x}) [1 - (x_1/g(\vec{x}))^2]$$

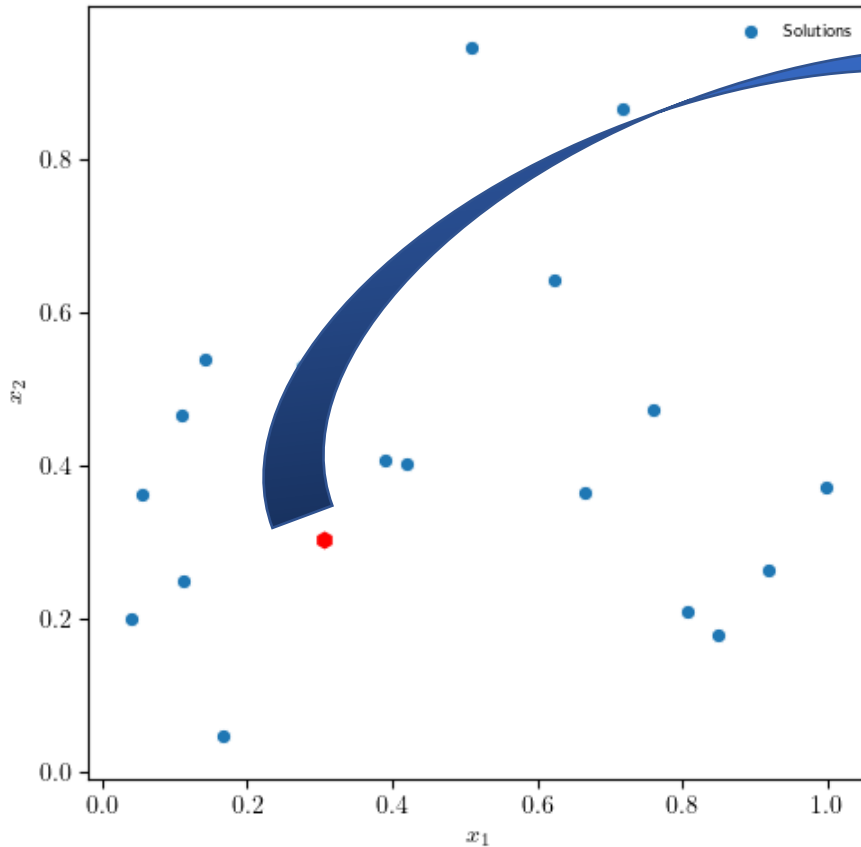
$$g(\vec{x}) = 1 + 10(n - 1) + \sum_{i=2}^n (x_i^2 - 10 \cos(4\pi x_i))$$

$$0 \leq x_1 \leq 1$$

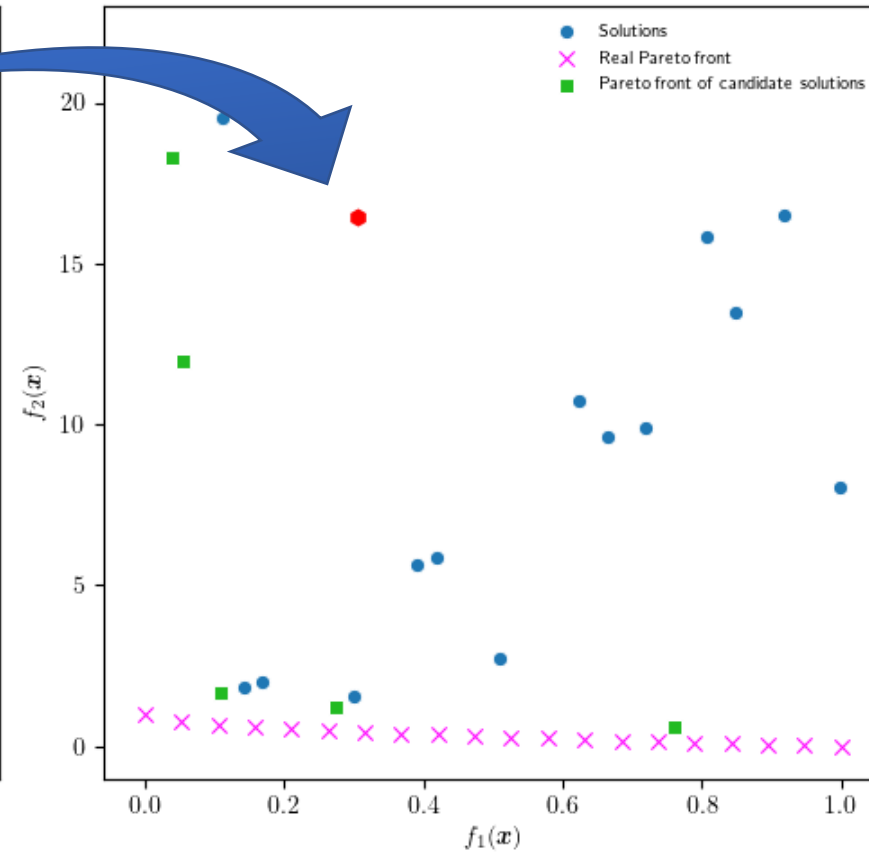
$$-5 \leq x_i \leq 5, i = 2, \dots, n$$

From solution space to objectives space

Solution space



Objectives space



ZDT-4 benchmark function

$$f_1(\vec{x}) = x_1$$

$$f_2(\vec{x}) = g(\vec{x}) [1 - (x_1/g(\vec{x}))^2]$$

$$g(\vec{x}) = 1 + 10(n - 1) + \sum_{i=2}^n (x_i^2 - 10\cos(4\pi x_i))$$

$$0 \leq x_1 \leq 1$$

$$-5 \leq x_i \leq 5, i = 1, \dots, n$$

Decreasing x_1 improves f_1
but **affects** f_2 !

Simple (HENCE WRONG) approaches to MOO

- How would YOU solve a problem with multiple fitness functions?

Simple (HENCE WRONG) approaches to MOO

- How would YOU solve a problem with multiple fitness functions?
- The first temptation would be to **sum all values** in a single fitness value
 - Problem 1: what if we are maximizing some objectives and minimizing some others?
 - Problem 2: what if there are huge differences in the magnitude of fitness values, i.e., some objectives can take huge values and some take tiny values?

Simple (HENCE WRONG) approaches to MOO

- How would YOU solve a problem with multiple fitness functions?
- The first temptation would be to **sum all values** in a single fitness value
 - Problem 1: what if we are maximizing some objectives and minimizing some others?
 - Problem 2: what if there are huge differences in the magnitude of fitness values, i.e., some objectives can take huge values and some take tiny values?
- Objectives weighting: optimize $Z = \sum_{i=1}^k w_i f_i(\mathbf{x})$ where $0 < w_i < 1$ and $\sum_{i=1}^k w_i = 1$
 - Problem 1: the **vector of weights influences the optimization**. How to choose such vector?
 - Problem 2: what if we **do not know** which values our objective functions will take?
 - Problem 3: what if the values of one or more objective functions span over **several orders of magnitude**, e.g., can range from $1e-10$ up to $1e10$?

Simple (not yet good) approaches TO MOO

Vector Evaluated Genetic Algorithm (VEGA)

- Performs multiple rounds of selection: one for each objective
- The new population is formed by using individuals selected for different objectives
- Problem: the algorithm suffers from speciation, **the population tends to converge to to sub-populations that are optimal for just a single objective**
- [\[Schaffer, 1984\]](#)

Simple (slightly better) approaches to MOO

Method of distance functions: optimize $Z = (\sum_{i=1}^k |f_i(\mathbf{x}) - \bar{y}_i|^r)^{\frac{1}{r}}$

- Where $1 \leq r < \infty$ (generally, $r = 2$ is used)
 - $\bar{\mathbf{y}} = (\bar{y}_1, \dots, \bar{y}_k)$ is the vector of **optimal fitness values**
 - It is basically the idea of minimizing the distance wrt optimal objective values
 - [\[Srinivas and Deb, Evol Comp 1994\]](#)
-
- Problem 1: a wrong selection of the vector of optimal values can lead to **non Pareto-optimal solutions** (i.e., there are better solutions that cannot be reached)
 - Problem 2: the vector $\bar{\mathbf{y}}$ is often unknown

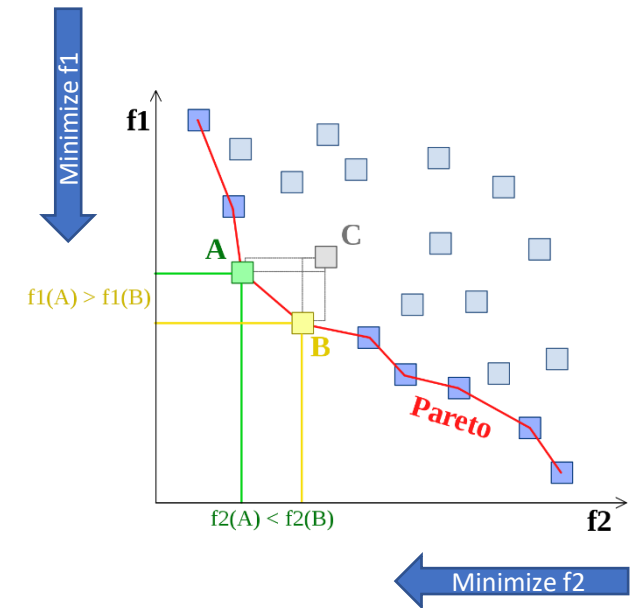
The hard truth

- Observation: all these methods collapse k objective functions to a single value $\mathbf{Z}$ and then return a single optimal solution
- **This is not multi-objective optimization:** all these methods are just reducing the problem to a single-objective optimization problem
- **We need to change our perspective** completely, beginning from realizing the fact that there can be multiple optimal solutions



Wait, what?

- Look at the figure: solution A is better than solution B wrt fitness f_2
- However, solution B is better than solution A wrt to fitness f_1
- This is a fact, and **we must acknowledge this situation**
- The use of weighted approaches is just WRONG



Domination

Any trivial recombination of the fitness values basically never works: we need to perform actual MOO

- Fact: there might be **infinite** optimal (and perfectly valid) solutions called **dominating solutions**

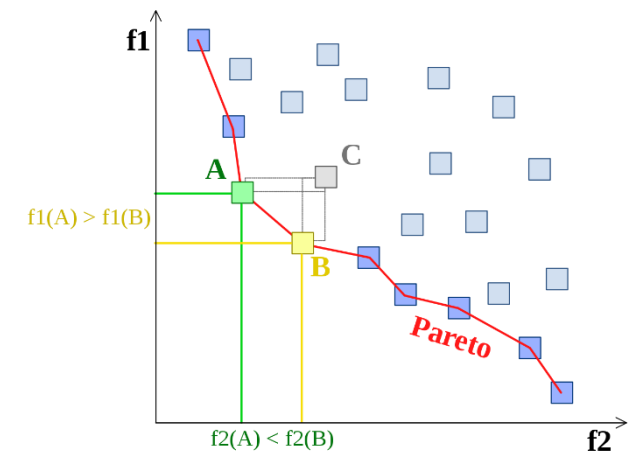
- A feasible solution $\mathbf{x}_1$ dominates another solution $\mathbf{x}_2$ iff:

$$f_i(\mathbf{x}_1) < f_i(\mathbf{x}_2) \quad \forall i \in \{1, 2, \dots, k\}$$

- A feasible solution $\mathbf{x}_1$ *weakly* dominates another solution $\mathbf{x}_2$ iff:

$$f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2) \text{ for at least one } j \in \{1, 2, \dots, k\}$$

- Dominance is denoted by $\mathbf{x}_1 \prec \mathbf{x}_2$
- A solution $\mathbf{x}^*$ is called Pareto-optimal if $\mathbf{x}^* \prec \mathbf{s}$ for any other solution $\mathbf{s}$ in the solution space
- Of course, domination works also in case of objectives maximization (just change to $>$ for those objectives)



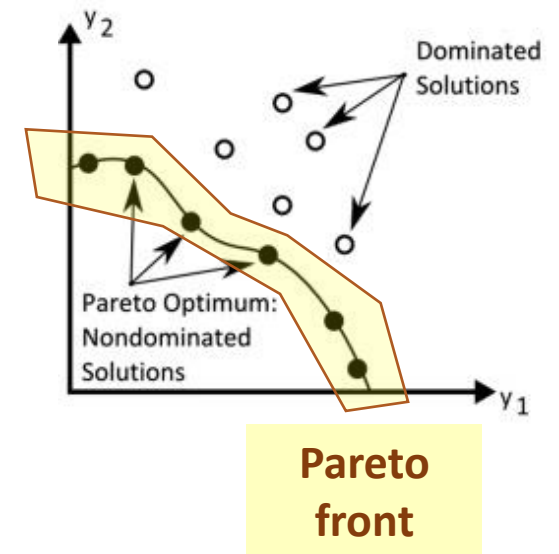
Pareto front

In MOO **there is basically never a single dominant solution**

- On the contrary, we look for the **set (front) of Pareto-dominant solutions**
- This set has (potentially) **infinite elements** (e.g., in case of real-valued solutions)

An **approximation** of the Pareto front would **still be good**

- The optimization method must «change» the individuals in the **search space** pushing to population towards the Pareto front in the **fitness space**!
- Possibly **preventing the convergence** to a degenerate population **collapsed around a single point**



Complexities of Pareto fronts

- A Pareto front can contain **infinite solutions**
- A Pareto front can also be **concave or discontinuous**
- **Its shape is unknown**, and it is **tricky to visualize** (especially when we have more than 2 objectives!)
- See [Angus and Woodward, Swarm Intell 2009]

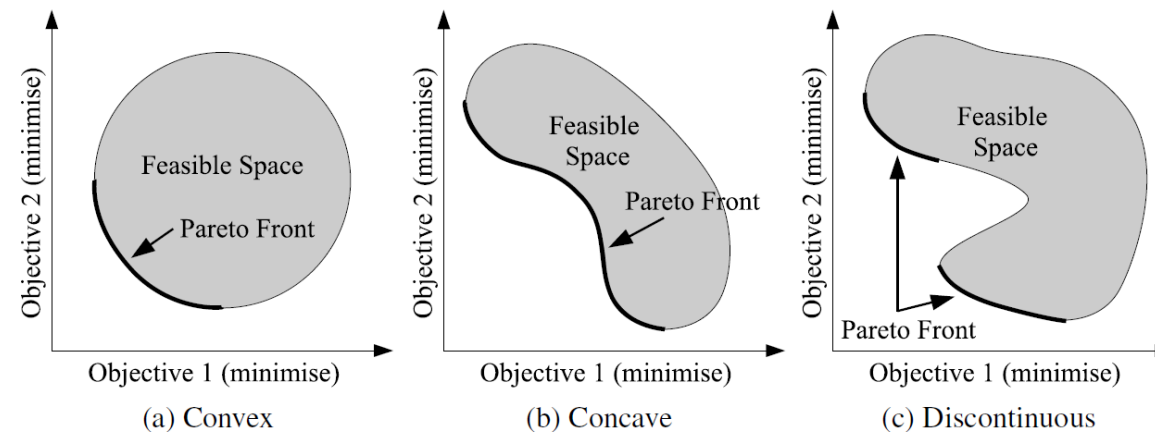


Fig. 1 Representations of different Pareto fronts plotted in the objective space

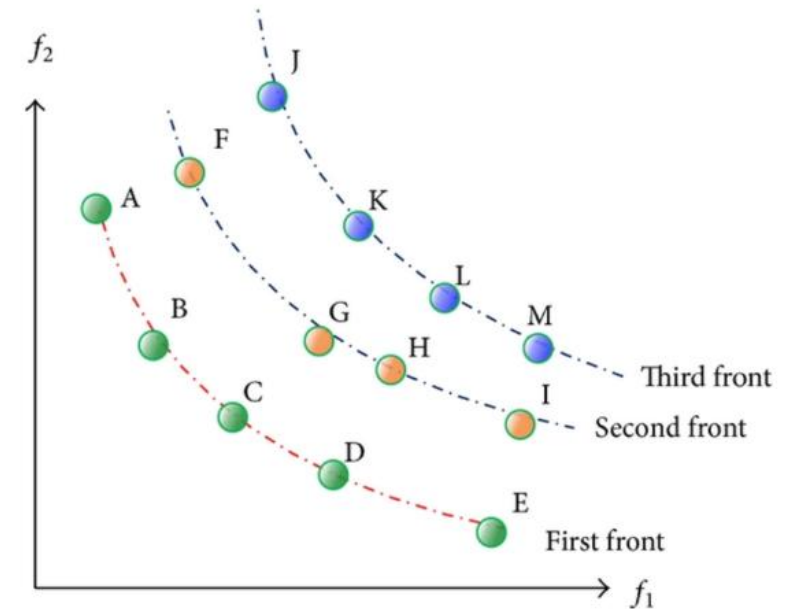
Non-dominated Sorting

Pareto-dominance induces a **partial-ordering of solutions**

- E.g., all solutions of the Pareto-optimal front have the same rank
- We can **sort the fronts** according to their dominance levels

It is possible to «peel out» the fronts, one by one:

1. $\mathcal{P}$ = population
2. $i = 0$
3. $i++$
4. $\mathcal{O}$ = Pareto-optimal front of $\mathcal{P}$
5. Save the individuals of $\mathcal{O}$ as $\mathcal{D}_i$
6. $\mathcal{P} = \mathcal{P} \setminus \mathcal{O}$
7. Repeat from 3 until $\mathcal{P} = \emptyset$



Issues in MOO

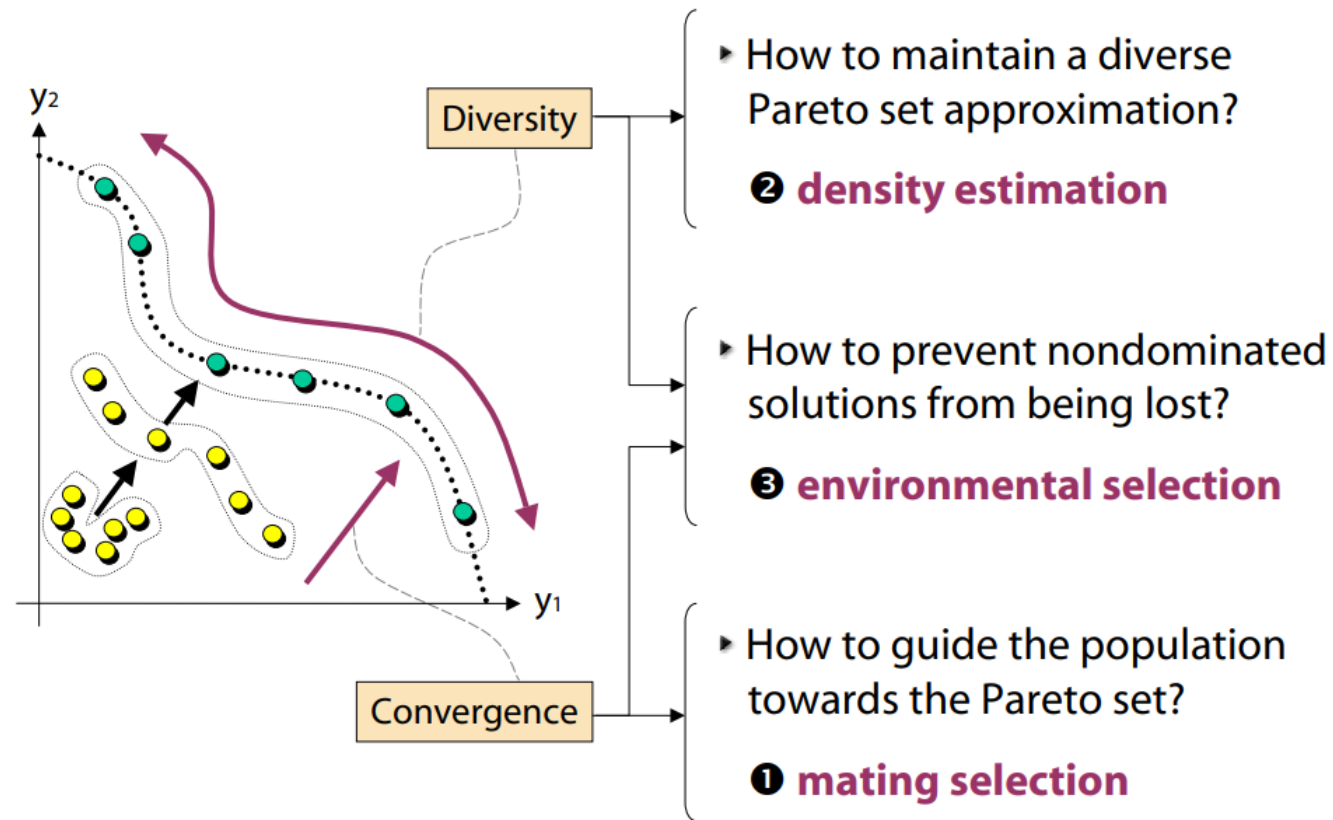


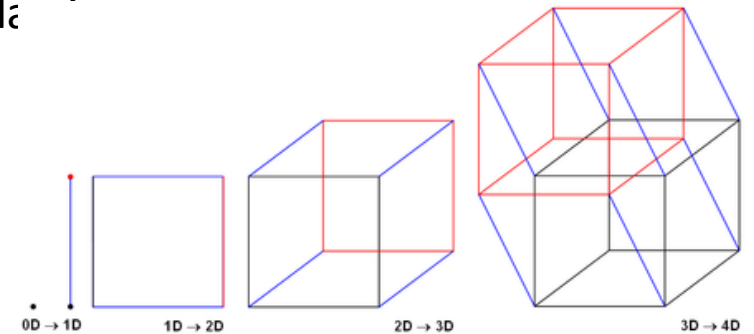
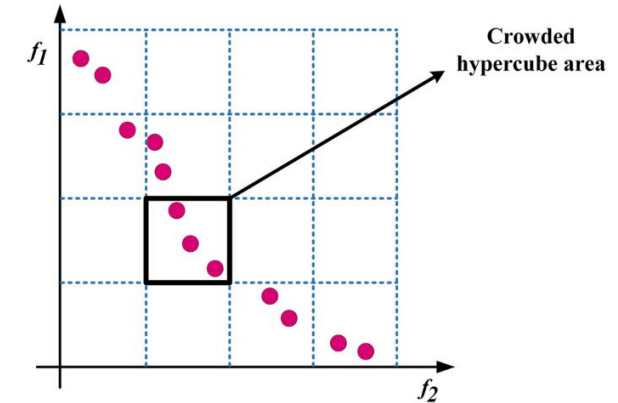
Figure from Eckart Zitzler's
"Tutorial on Evolutionary Multiobjective Optimization", MOMH 2002

MOPSO [1/3]

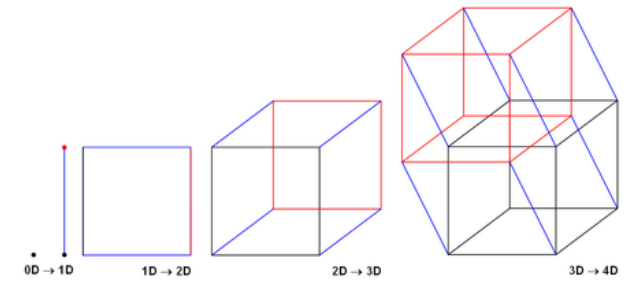
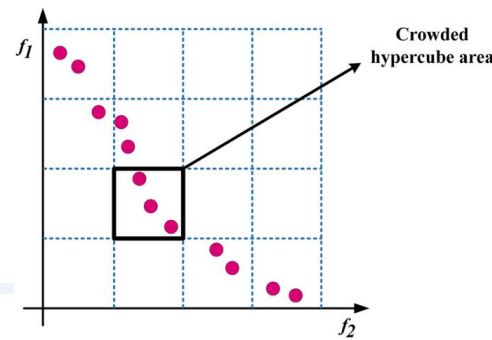
How to perform MOO? One option is to use **Swarm Intelligence (PSO)**

Multi-Objective Particle Swarm Optimization (MOPSO)

- Swarm Intelligence meta-heuristic for multi-objective optimization
- Idea 1 (convergence) **change the particles' velocity update** with respect to classical PSO and let particles be **attracted «towards the Pareto front»**
- That is, **towards the dominating particles** found so far
- Idea 2 (diversity) maintain a list of **hyper-cubes** (in objective space) that keep track of the dominating positions visited so far, and **pick attractors from less crowded hyper-cubes**
- [Coello-Coello and Lechuga, IEEE CEC 2002]



MOPSO [2/3]



- In MOPSO, a global **repository of non-dominated particles** *REP* is maintained
 - When a new dominating solution is added to *REP*, the dominated particles are removed
 - The repository has a **limited size** Q
 - IF the number of particles in the repository $> Q$ THEN the particles are mapped into the hyper-cubes; **random particles of most crowded hyper-cubes are removed** until the repository contains exactly Q particles
- MOPSO's velocity update formula becomes
$$\mathbf{v}_i(t+1) = w \cdot \mathbf{v}_i(t) + c_{soc} \cdot \mathbf{r}_1 \otimes (\mathbf{REP}_h - \mathbf{x}_i(t)) + c_{cog} \cdot \mathbf{r}_2 \otimes (\mathbf{p}_i - \mathbf{x}_i(t))$$
 - $\mathbf{REP}_h$ is the h -th particle of the repository
 - The index h is determined using a **roulette wheel selection over the hyper-cubes** (less crowded = higher probability) and then randomly picking a particle in the hyper-cube
 - The personal best $\mathbf{p}_i$ now corresponds to the best position visited so far by the i -th particle wrt all objectives, i.e., it is not updated when $\mathbf{p}_i < \mathbf{x}_i(t)$

MOPSO [3/3]

- MOPSO has **several hyper-parameters** to be tuned
 - The size of the repository, the size of the tournament, the size of hyper-cubes (i.e., the number of divisions), the size of the swarm, the inertia factor, the social and cognitive factors, the maximum (and minimum) velocity, the number of iterations...
- Some of the aforementioned settings can be derived by classic PSO but, once again, the optimal settings are **problem-dependent**
- **Adaptive methods have been proposed**, e.g., pccsAMOPSO [Hu and Yen, IEEE Tran Evol Comp 2013], AMOPSO [Han et al., IEEE Tran Cybern 2017], AGMOPSO [Han et al., IEEE Tran Cybern 2017]

OMOPSO [1/2]

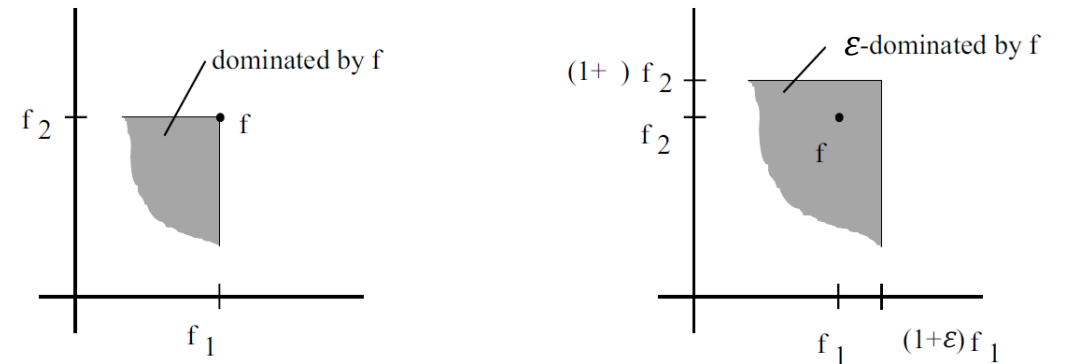
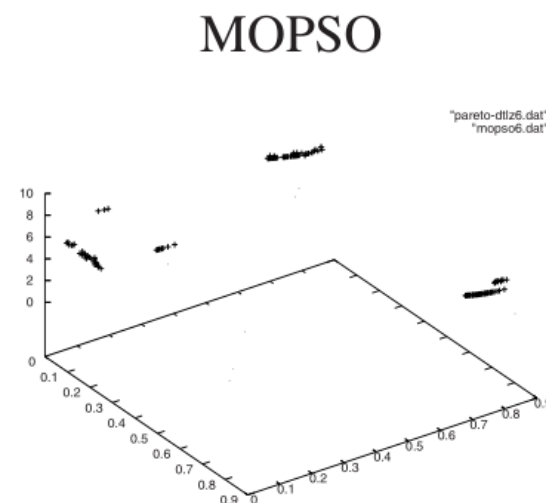
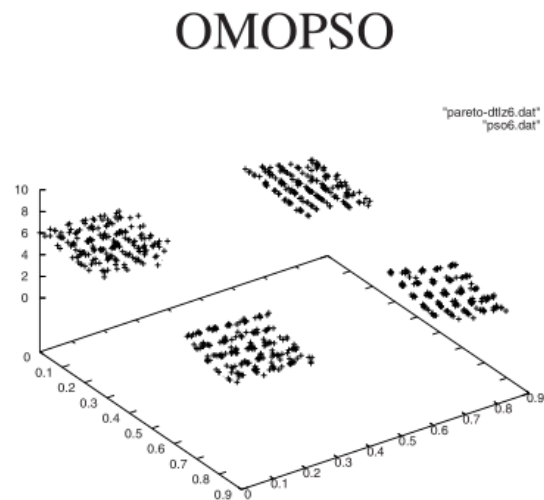


Figure 2: Graphs visualizing the concepts of dominance (left) and ϵ -dominance (right).

- **Extends MOPSO** by adding **mutation operators** and **ϵ -dominance**
 - [Sierra and Coello-Coello, EMO 2005]
- The swarm is partitioned into **multiple subgroups**: each subgroup uses a different **mutation operator**
 - Uniform operator promotes exploration, non-uniform operator promotes exploitation
- **ϵ -dominance** (relaxed dominance) is used to **update the repository of dominating particles** (reduces crowding)

OMOPSO [2/2]

- PRO: **better distribution** of solutions on the Pareto front with respect to classic MOPSO
- CON: **additional hyper-parameter** (ε) with relevant impact to performance!
- **Adaptive methods exist** (e.g., increase ε when the Pareto front has too many solutions)



NSGA-II

Extends GAs to **MOO**

- Uses **elitism**
- Uses **fast dominated sorting** to **rank** solutions
- Leverages **Crowding Distance** (CD) to maintain diversity in the population
- **Tournament selection**: best ranked individuals win; if individuals have the same rank, CD is used

A fast and elitist multiobjective genetic algorithm: **NSGA-II**

[K Deb](#), [A Pratap](#), [S Agarwal](#)... - IEEE transactions on ..., 2002 - ieeexplore.ieee.org

... **NSGA-II**. From the simulation results on a number of difficult test problems, we find that

NSGA-II ... In this paper, we suggest a simple constraint-handling strategy with **NSGA-II** that suits ...

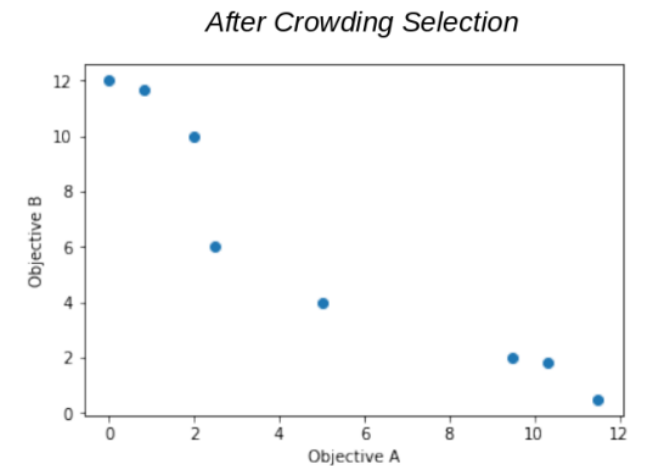
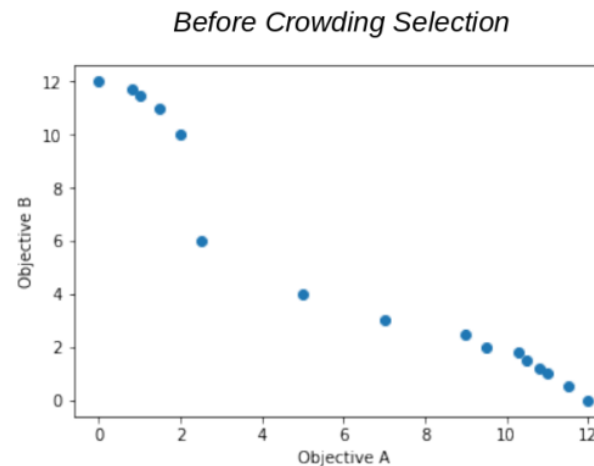
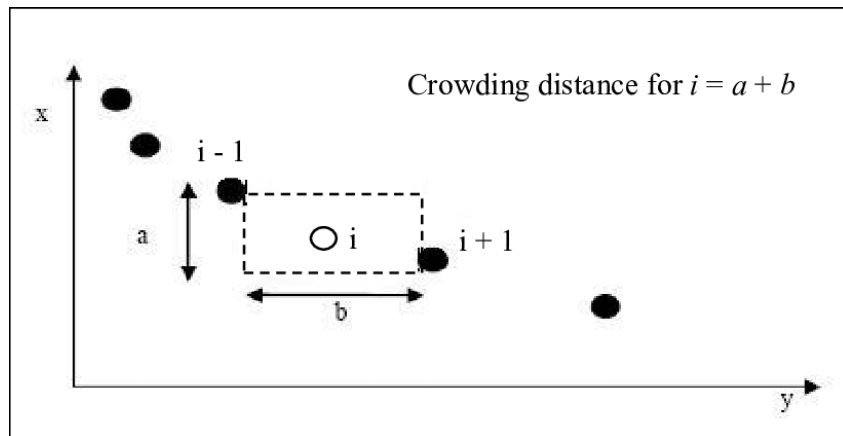
☆ Salva Cita Citato da 58975 Articoli correlati Tutte e 41 le versioni Web of Science: 35123



Crowding Distance (CD)

To prevent the loss of diversity we need a «measure» of diversity

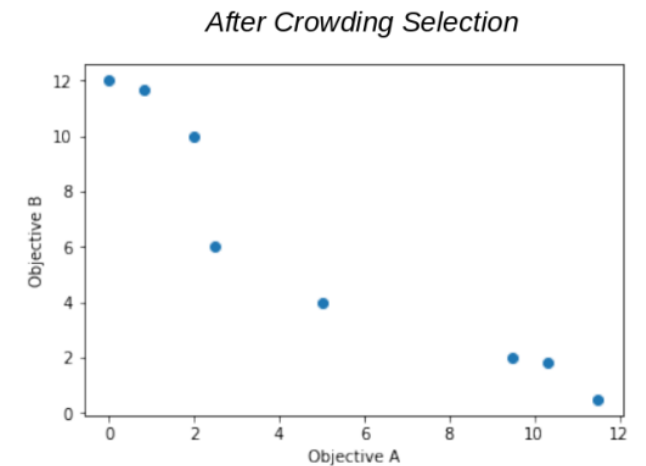
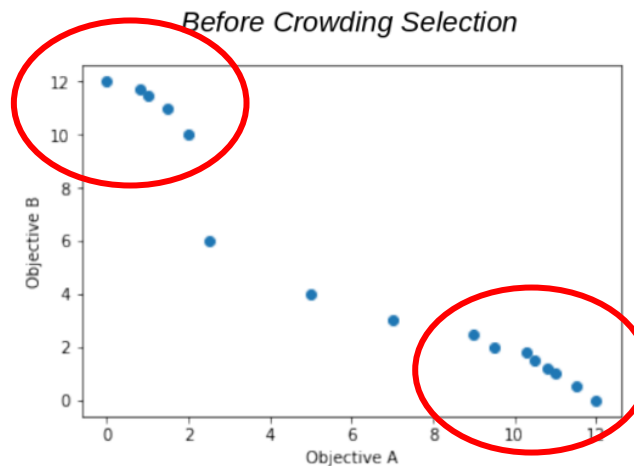
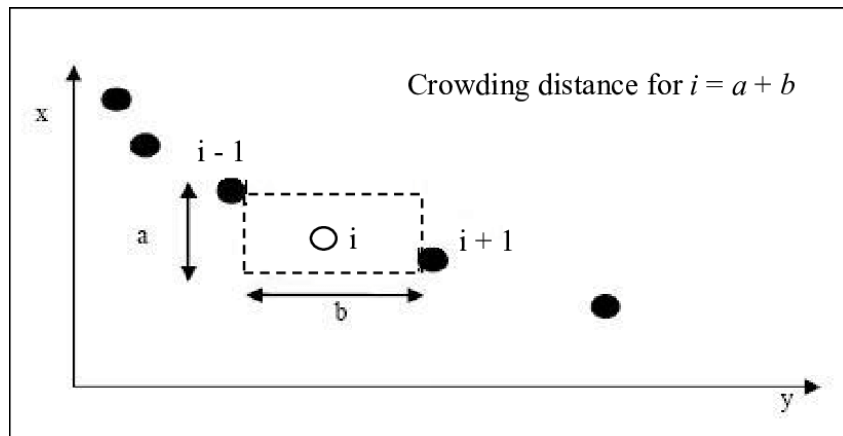
- Diversity in **objective space**
- **Crowding Distance (CD)** is one example of this measure
- CD-based selection: less crowded solutions (belonging to the *same* front) have a **higher possibility of being selected** for the next generation



Crowding Distance (CD)

To prevent the loss of diversity we need a «measure» of diversity

- Diversity in **objective space**
- **Crowding Distance (CD)** is one example of this measure
- CD-based selection: less crowded solutions (belonging to the *same* front) have a **higher possibility of being selected** for the next generation

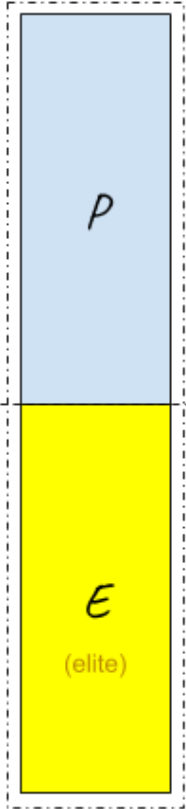


Crowded-comparison operator

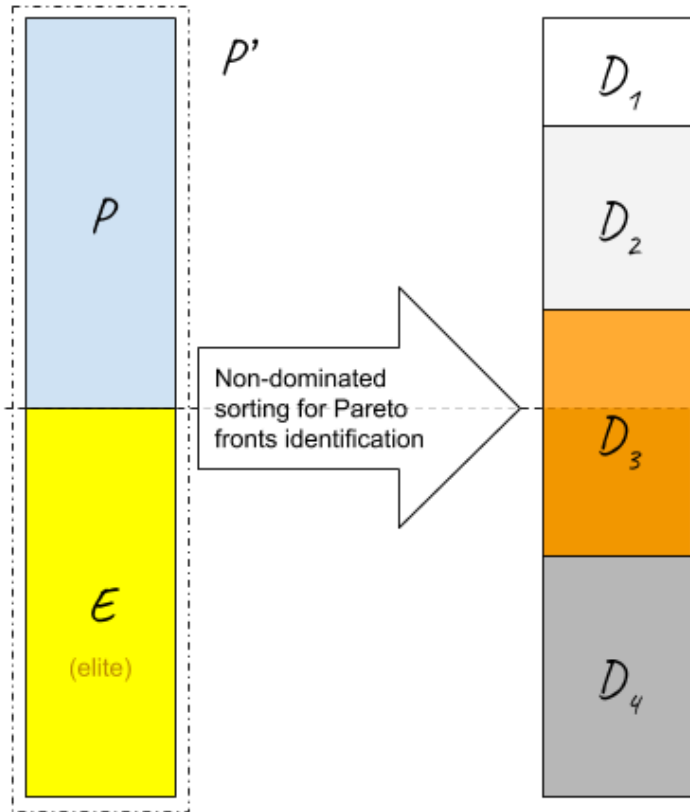
The **crowded-comparison operator**, denoted by $\prec_n$, guides the selection process

- Designed to obtain a uniformly spread out the Pareto front
- Assume that each individual of the population has a rank R_i and a crowding distance CD_i
 - if $R_i < R_j$ then $i \prec_n j$ (usual domination rank)
 - if $R_i = R_j$ and $CD_i > CD_j$ then $i \prec_n j$ (same rank $\rightarrow$ larger crowding distance is preferred)
- $\prec_n$ induces a **partial order** on the solutions
- [Deb et al., IEEE Tran Evol Comp 2002]

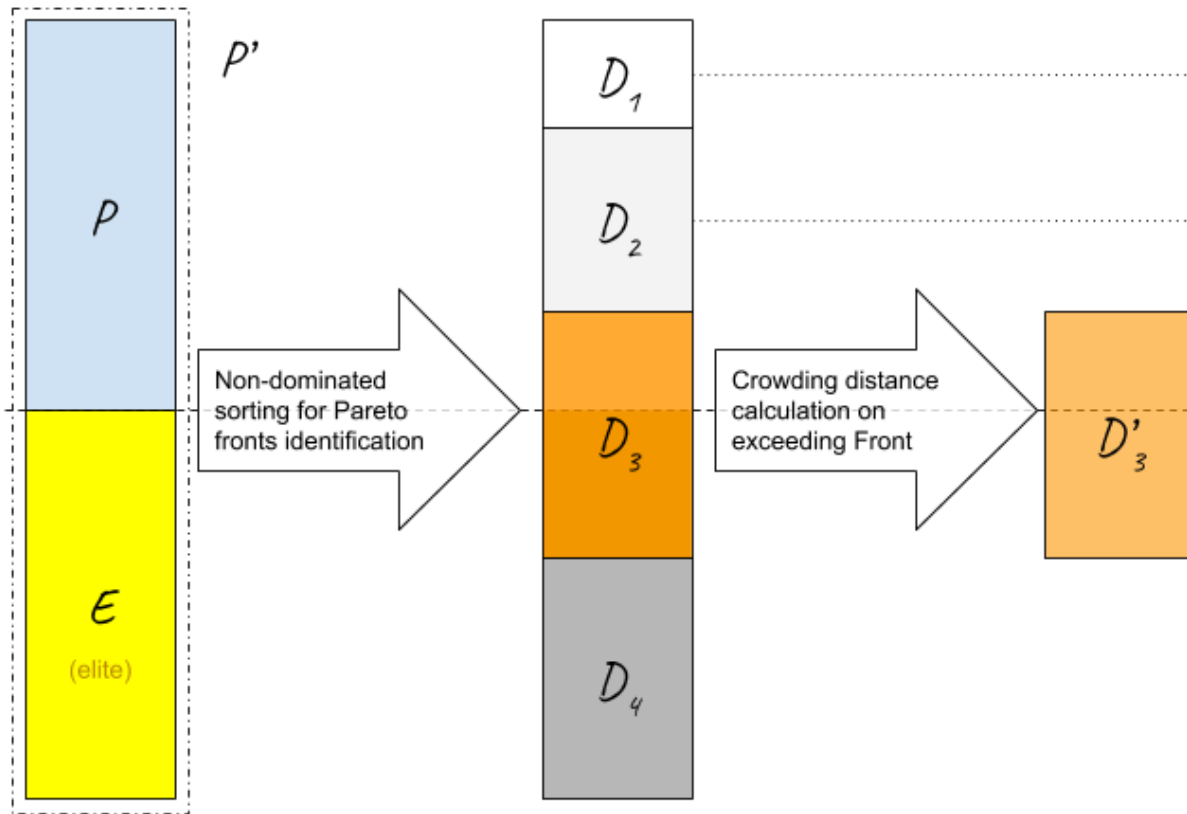
NSGA-II algorithm



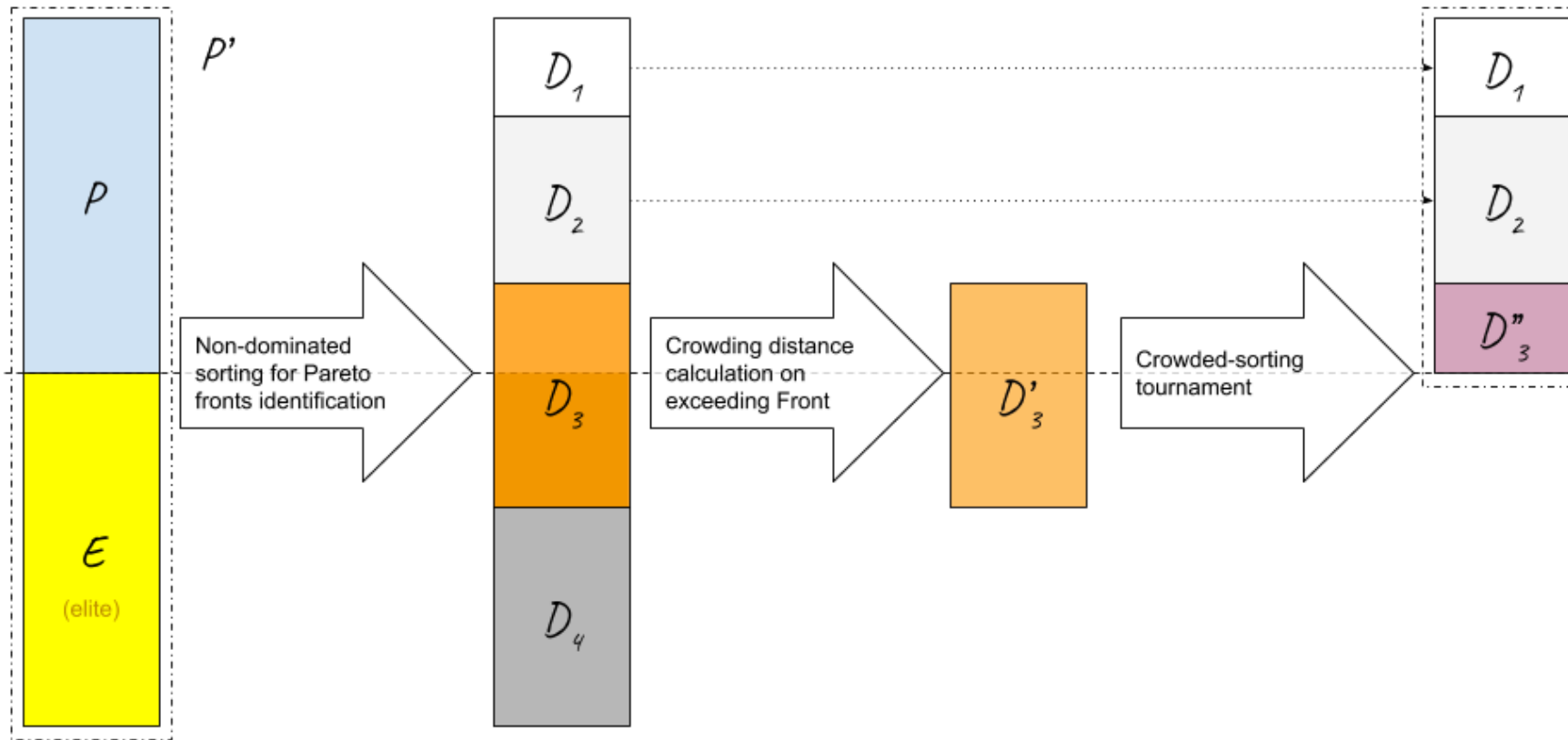
NSGA-II algorithm



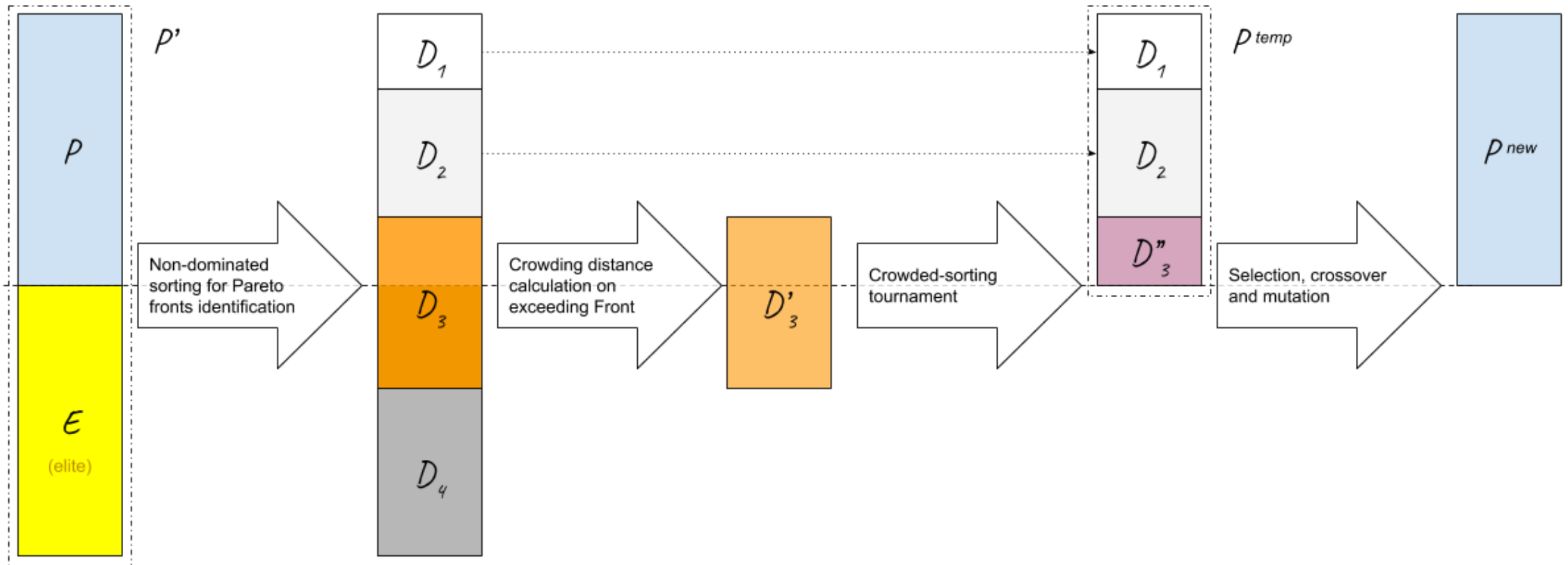
NSGA-II algorithm



NSGA-II algorithm



NSGA-II algorithm



Applied NSGA-II

ORIGINAL RESEARCH article

Front. Genet., 31 March 2021

Sec. Computational Genomics

Volume 12 - 2021 | <https://doi.org/10.3389/fgene.2021.617935>

This article is part of the Research Topic

Network-Oriented Approaches to Anticancer Drug Response

[View all 8 Articles >](#)

Screening for Combination Cancer Therapies With Dynamic Fuzzy Modeling and Multi-Objective Optimization



Simone Spolaor¹



Martijn Scheve²

Murat Firat²



Paolo Cazzaniga^{3,4,5}



Daniela Besozzi^{1,4,5}



Marco S. Nobile^{2,4,5*}

¹ Department of Informatics, Systems and Communication, University of Milano-Bicocca, Milan, Italy

² Department of Industrial Engineering & Innovation Sciences, Eindhoven University of Technology, Eindhoven, Netherlands

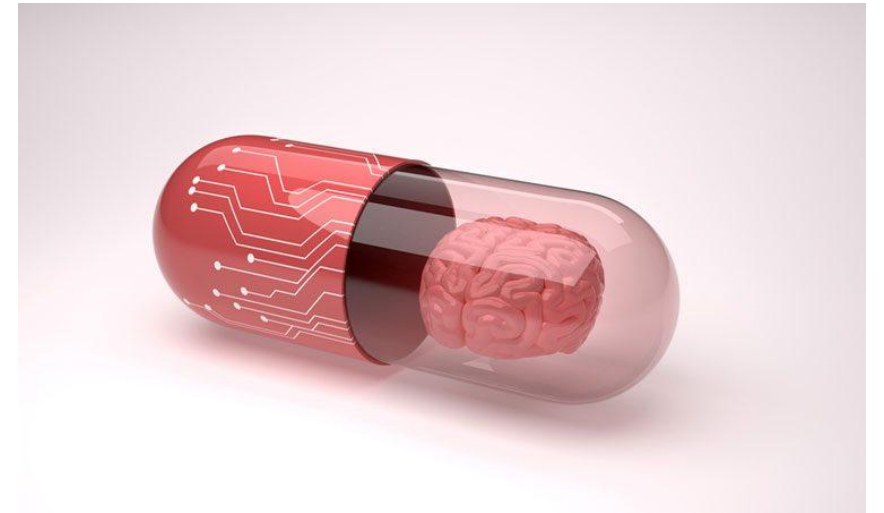
³ Department of Human and Social Sciences, University of Bergamo, Bergamo, Italy

⁴ SYSBIO/ISBE.IT Research Centre of Systems Biology, Milan, Italy

⁵ Bicocca Bioinformatics Biostatistics and Bioimaging Centre (B4), Milan, Italy

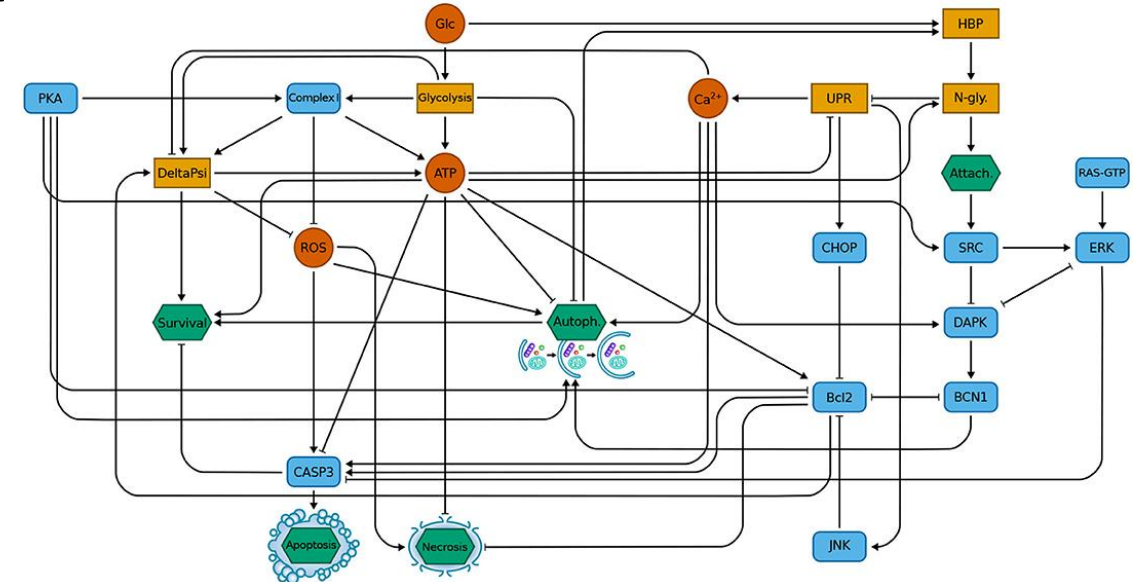
Goal: create a new drug for breast cancer

- How? by combining multiple molecules
- First goal: **maximize apoptosis** in cancer cells
- Second goal: **minimize necrosis** in cancer cells
- Third goal: **minimize complexity** of treatment
- MOO problem: determine **which molecules** should be used in the drug



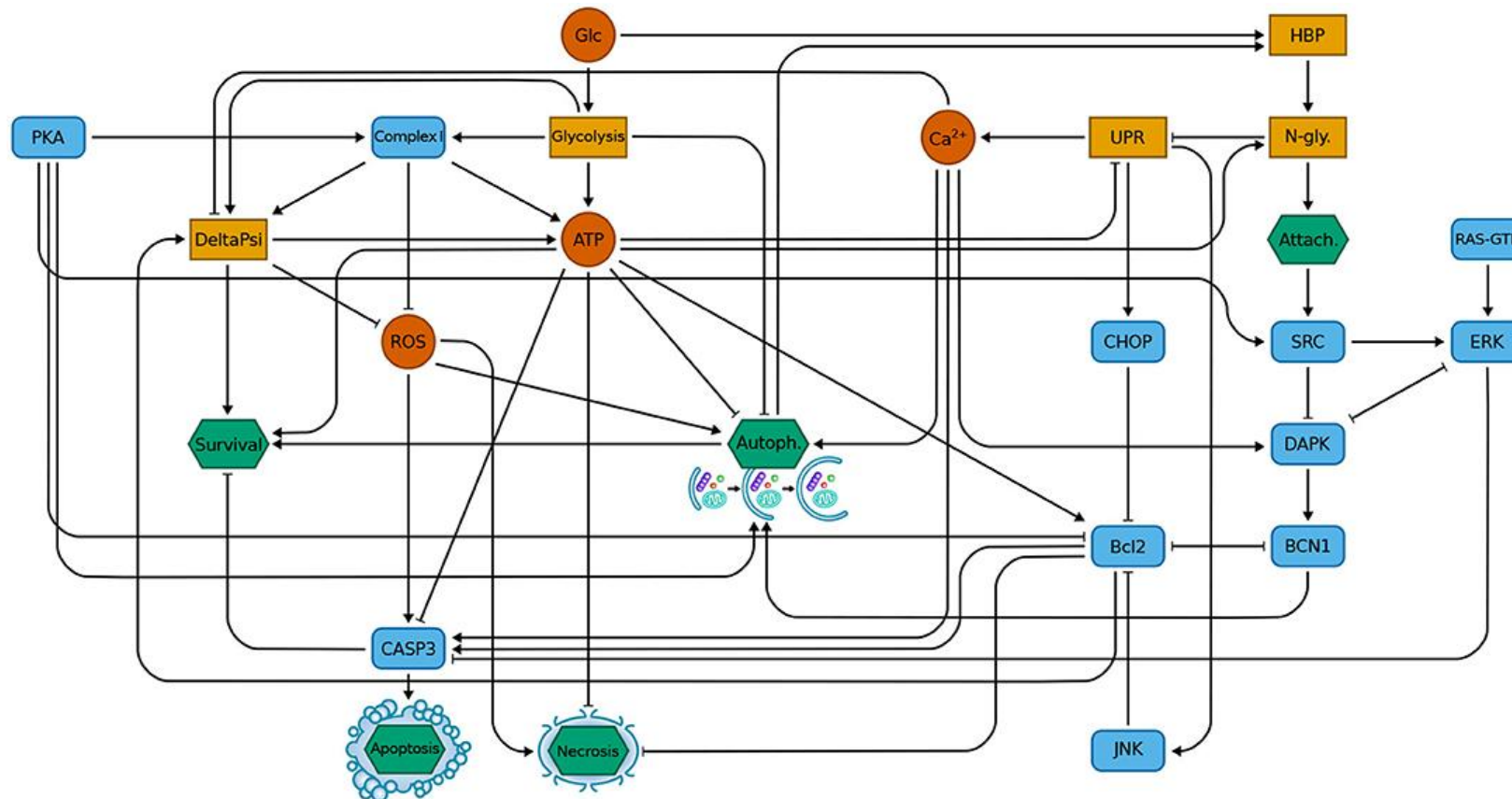
Dynamic fuzzy modeling

- We created a FIS to create a **phenomenological dynamic model of the functioning of cancer cells**
 - **>200 rules**, each based on a specific scientific paper
 - Example of rule: «IF glucose is high THEN Glycolysis is high»
 - **Network** of interacting linguistic variables: **complex and unpredictable dynamics**
 - **0-order Sugeno inference**
 - Several experiments for **validation**
 - [Nobile et al., Bioinformatics 2020],
[Nobile et al., Front Genet 2021]
 - NSGA-II algorithm to **identify optimal drug combinations**
-
- ```
graph TD; Glc((Glc)) --> Glycolysis[Glycolysis]; PKA[PKA] --> Glycolysis; PKA --> DeltaPsi[DeltaPsi]; PKA --> CASP3[CASP3]; Glycolysis --> ATP((ATP)); Glycolysis --> Complex[Complex]; Complex --> ATP; Complex --> ROS((ROS)); ATP --> ROS; ATP --> Survival{{Survival}}; ATP --> Autophagy{{Autophagy}}; ATP --> CASP3; ROS --> Survival; ROS --> Autophagy; ROS --> CASP3; Survival --> Autophagy; Ca2+((Ca2+)) --> Glycolysis; Ca2+ --> Autophagy; Ca2+ --> CASP3; DeltaPsi --> Survival; DeltaPsi --> Autophagy; DeltaPsi --> CASP3;
```



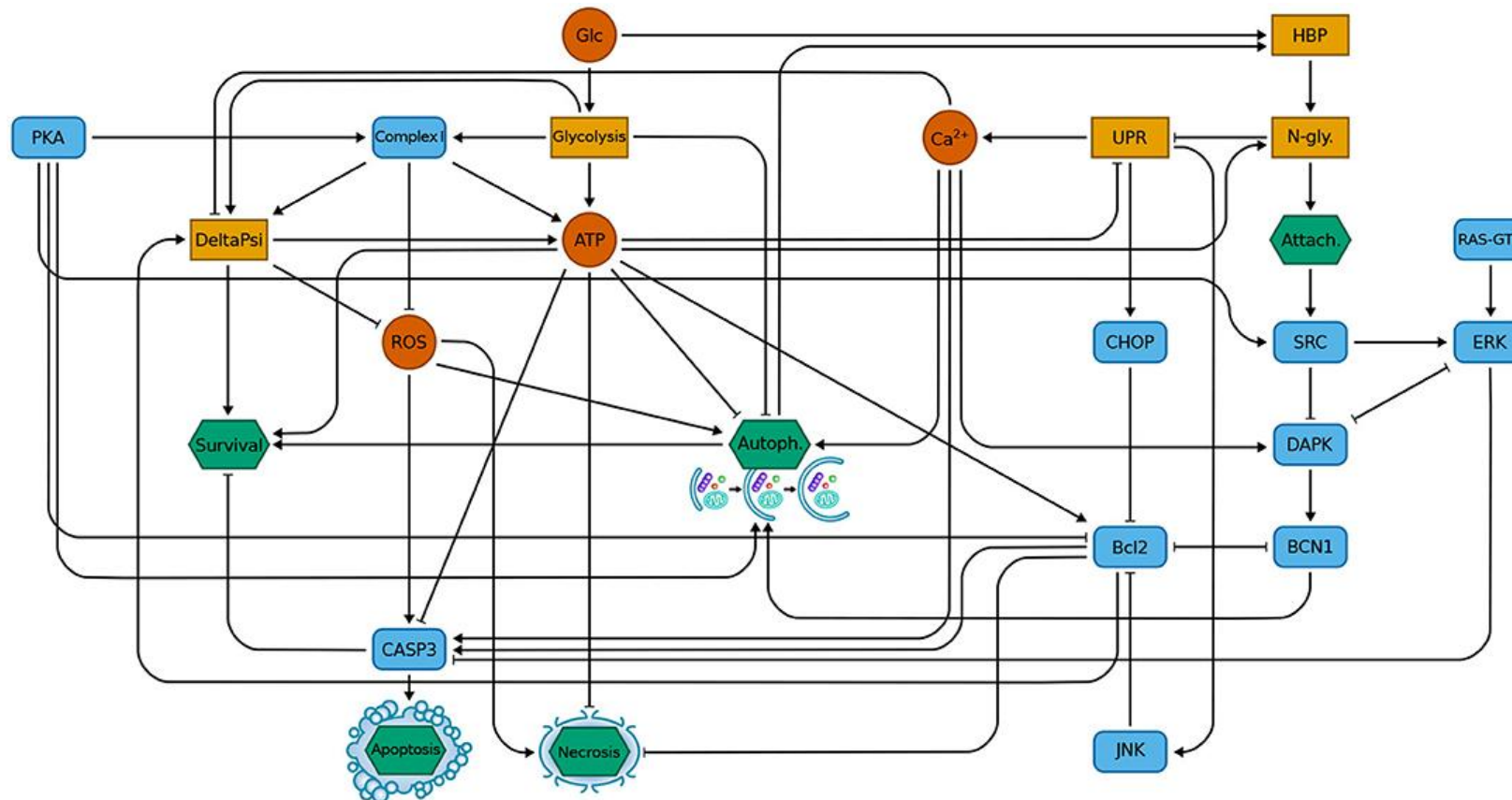
# How?

- NSGA-II + dynamic fuzzy modeling



# How?

- NSGA-II + dynamic fuzzy modeling



Upregulate

Downregulate

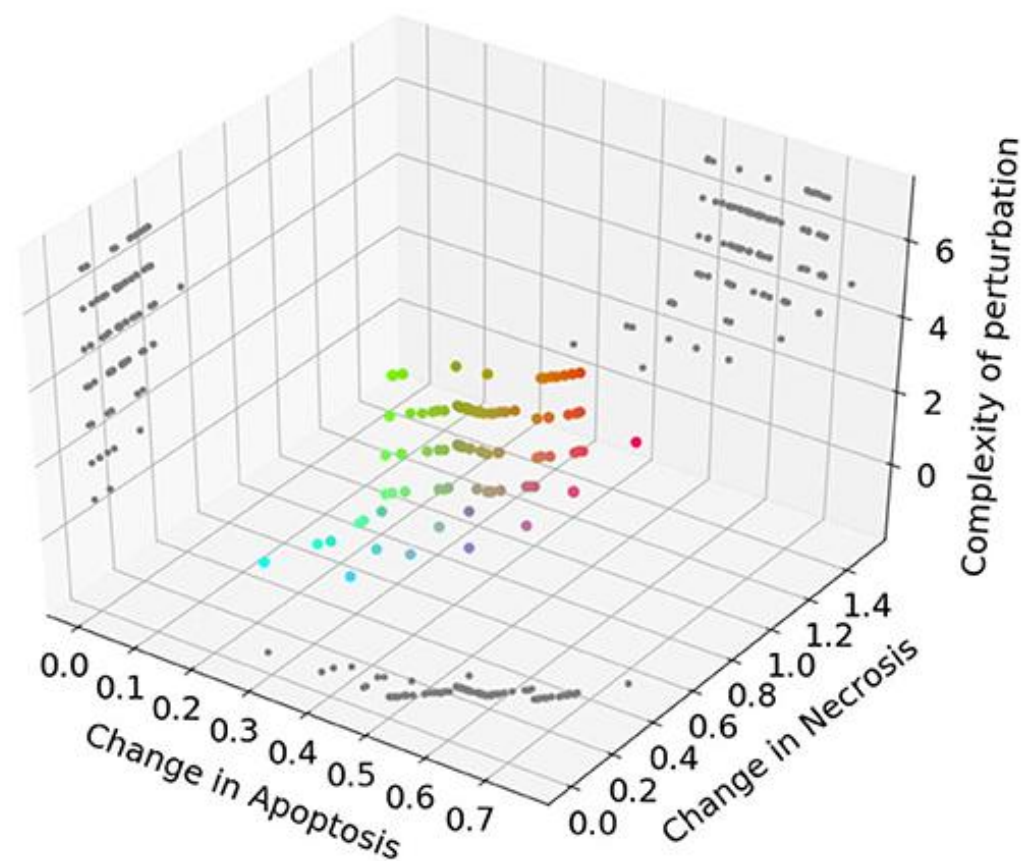
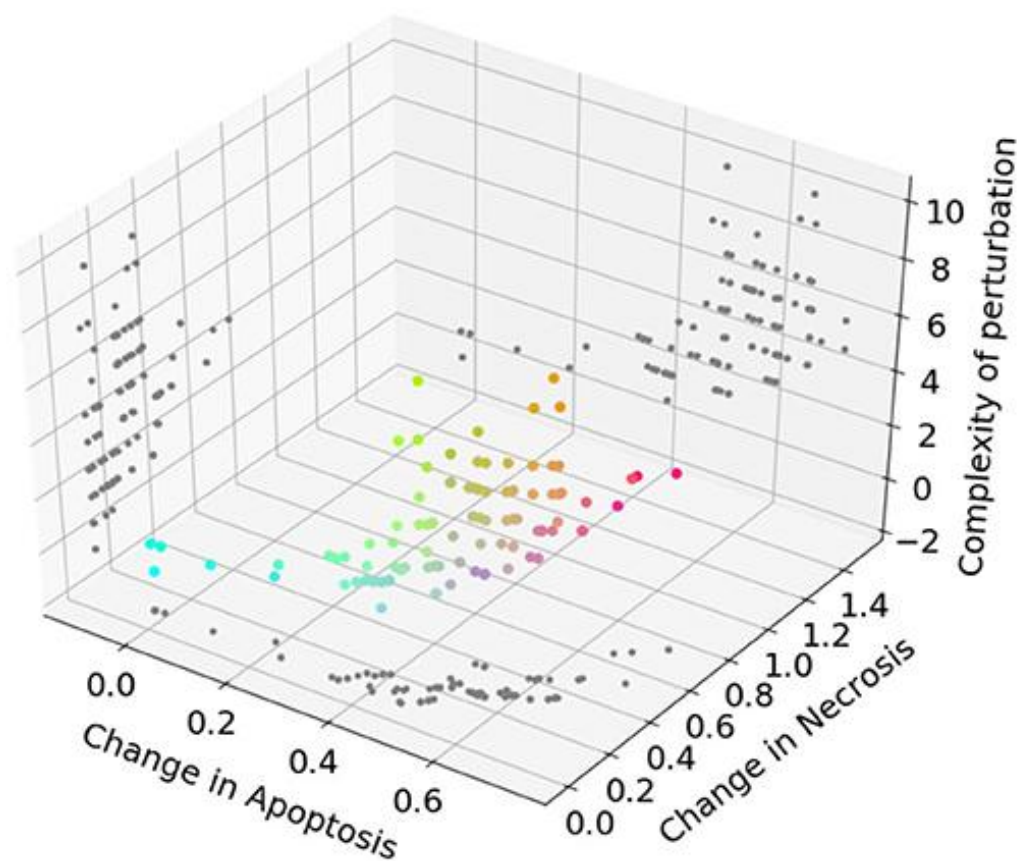
# NSGA-II: representation

---

- 3 possible states for each molecule (0: untouched, 1:  $\uparrow$ , 2:  $\downarrow$ )
- Find the vector of regulations that
  - maximizes apoptosis
  - minimizes necrosis and
  - minimizes the number of 1s and 2s

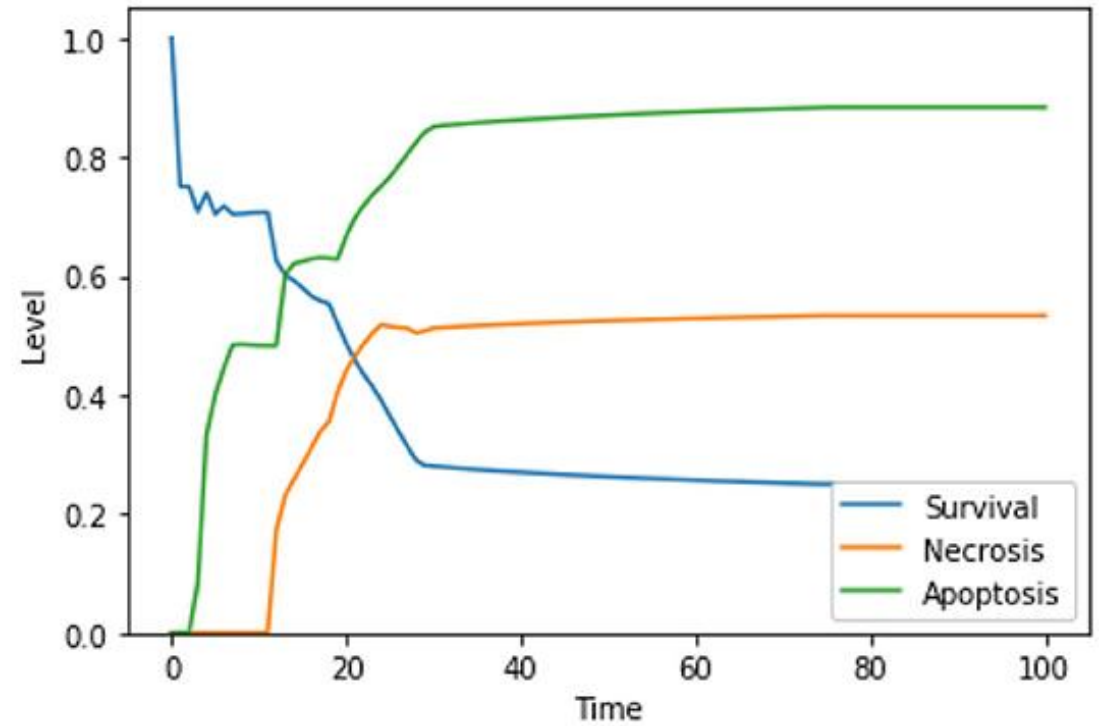
# Results: 3D Pareto front

---



# Our favourite solution

CI IS low, ERK IS low, UPR IS high





# NSGA-III

Extends NSGA-II to **many-objectives optimization** (>2 objs.)

- [\[Deb and Jain, IEEE Trans Evol Comput 2014\]](#)
- Different selection method for the exceeding front by means of “**reference points**”

NSGA-III's new selection method using reference points:

- **Reference points** are positions on a **normalized hyper-plane** in the **objective-space**
- **Reference line**: a line passing through the origin and a reference point
- Each solution is associated to the closer reference line
- **The solutions most close** to the reference lines are **selected**

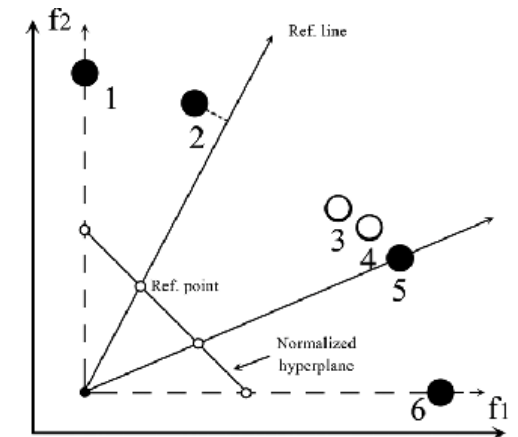


Figure from [Seada and Deb, 2014]

PRO: NSGA-III yields a more uniformly distributed Pareto front than NSGA-II; works well also in the case of many-objective problems

CON: reference points to be chosen, normalization in objective space is not trivial %

## Another example: drug discovery



# Objectives for drug discovery problem

---

- You aim at **interfering with some molecule** (e.g., protein, membrane receptor)
  - **You know the structure of the ligand** and its binding sites
  - You want to find the **optimal molecule that binds to that ligand**
- When you develop a new drug, you want to achieve **several goals**, notably:
  - Maximize synthesizability
  - Maximize on-target affinity
  - Minimize off-target affinity (all of them, in principle!)
  - Minimize costs
  - Optimize pharmacokinetics
  - Minimize toxicity
  - Etc.

# Objectives for drug discovery problem

---

- You aim at **interfering with some molecule** (e.g., protein, membrane receptor)
  - **You know the structure of the ligand** and its binding sites
  - You want to find the **optimal molecule that binds to that ligand**
- When you develop a new drug, you want to achieve **several goals**, notably:
  - **Maximize synthesizability**
  - **Maximize on-target affinity**
  - **Minimize off-target affinity** (all of them, in principle!)
  - Minimize costs
  - Optimize pharmacokinetics
  - Minimize toxicity
  - Etc.

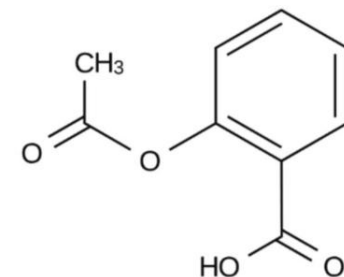
# Objectives for drug discovery problem

---

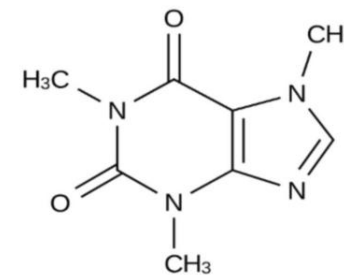
- You aim at **interfering with some molecule** (e.g., protein, membrane receptor)
  - **You know the structure of the ligand** and its binding sites
  - You want to find the **optimal molecule that binds to that ligand**
- When you develop a new drug, you want to achieve **several goals**, notably:
  - Maximize synthesizability
  - Maximize on-target affinity
  - Minimize off-target affinity (all of them, in principle!)
  - Minimize costs
  - Optimize pharmacokinetics
  - Minimize toxicity
  - Etc.

# Representation: SMILES

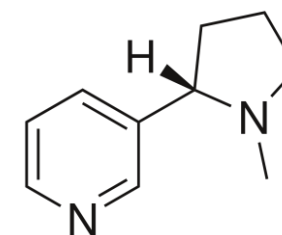
- «**Simplified Molecular Input Line Entry System**»
  - Strings that describe chemical species
  - Can be converted to 2D diagrams or 3D structure
- A SMILE describes the **atoms**, the **bonds** between atoms, the **branches**, the **rings**, the aromaticity, etc.
- The **GA** will explore the space of SMILE strings



**Acetylsalicylic Acid (Aspirin)**  
CC(=O)Oc1ccccc1C(=O)O



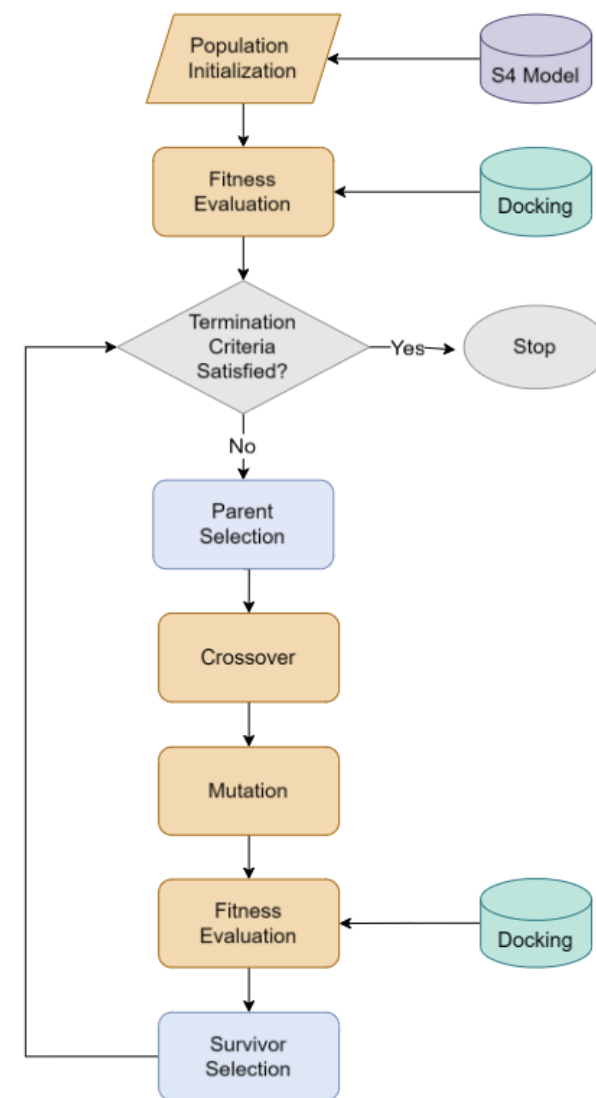
**Epinephrine**  
CNC[C@H](O)c1ccc(O)c(O)c1



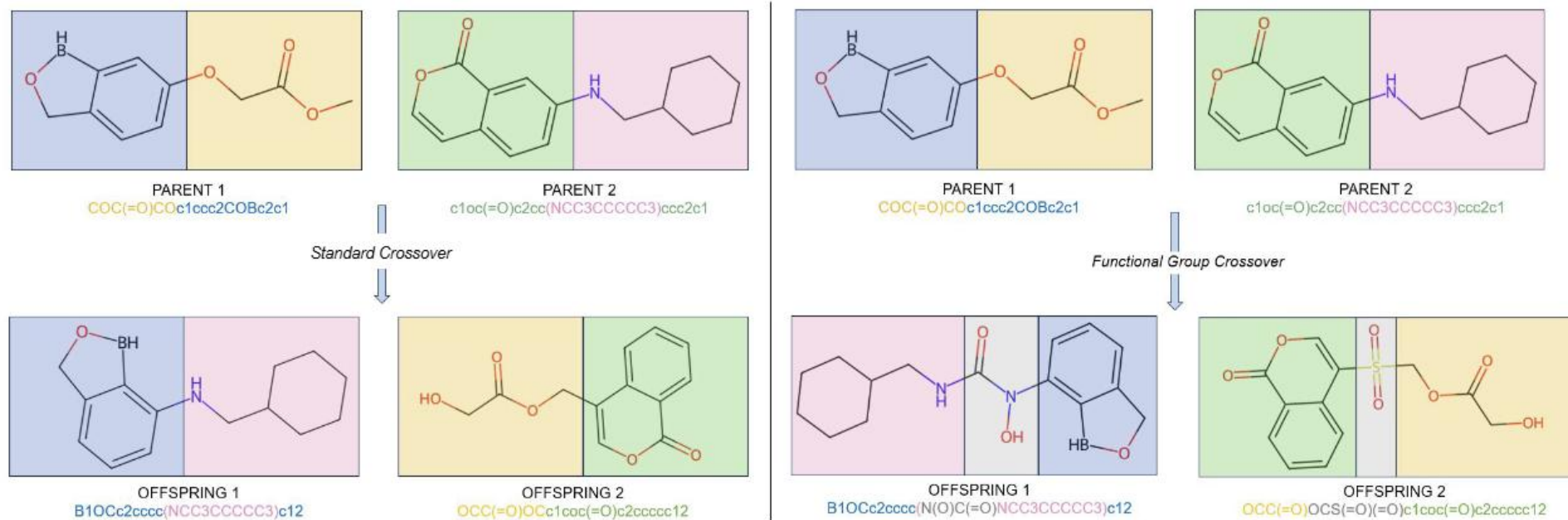
**Nicotine**  
CN1CCC[C@H]1c2cccnc2

# DRAGON

- **DRAGON:** DRugs Adaptation using Genetic Optimization and Neural-Networks *[paper in submission]*
- A chemical language model (S4) is used to implement a **generator of valid smiles**: we use that to create the **initial population**
- Then, we use **NSGA-III** to find the Pareto front

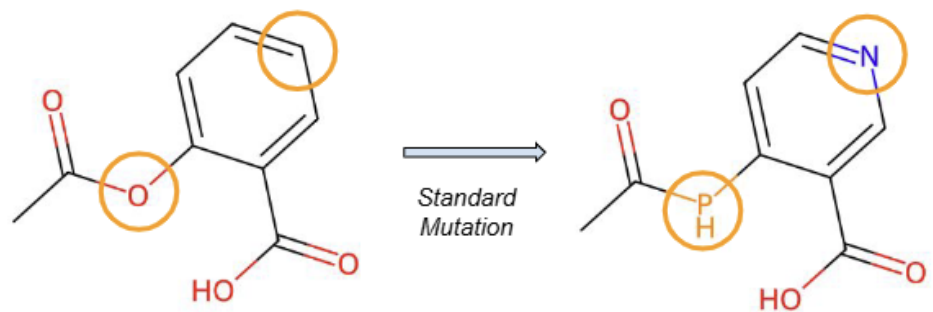


# Standard crossover vs functional group crossover



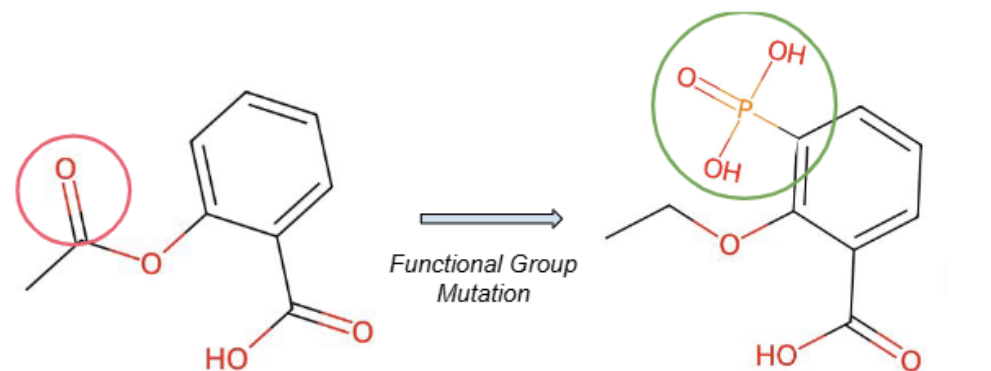


# Mutation vs Functional Group Mutation



ORIGINAL MOLECULE  
CC(=O)Oc1ccc(O)c(C(=O)O)c1

MUTATED MOLECULE  
CC(=O)P(=O)(O)Oc1ccc(O)c(C(=O)O)c1

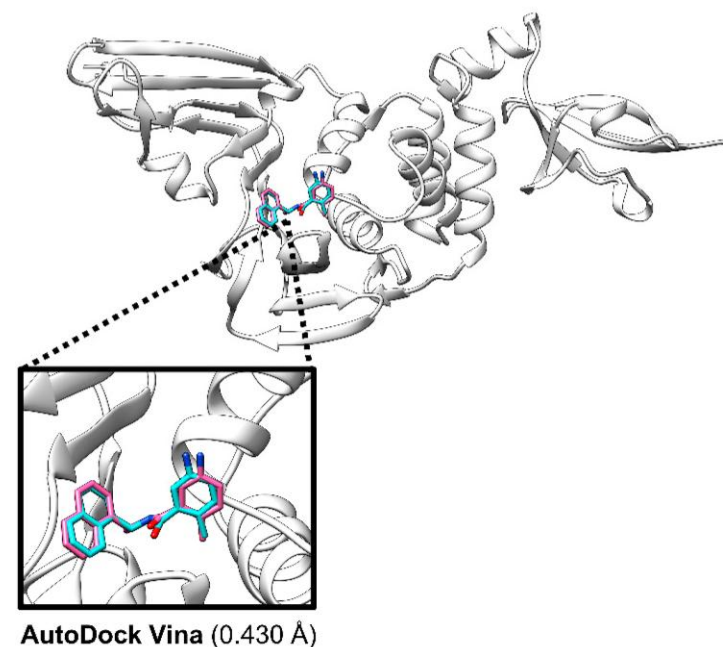


ORIGINAL MOLECULE  
CC(=O)Oc1ccc(O)c(C(=O)O)c1

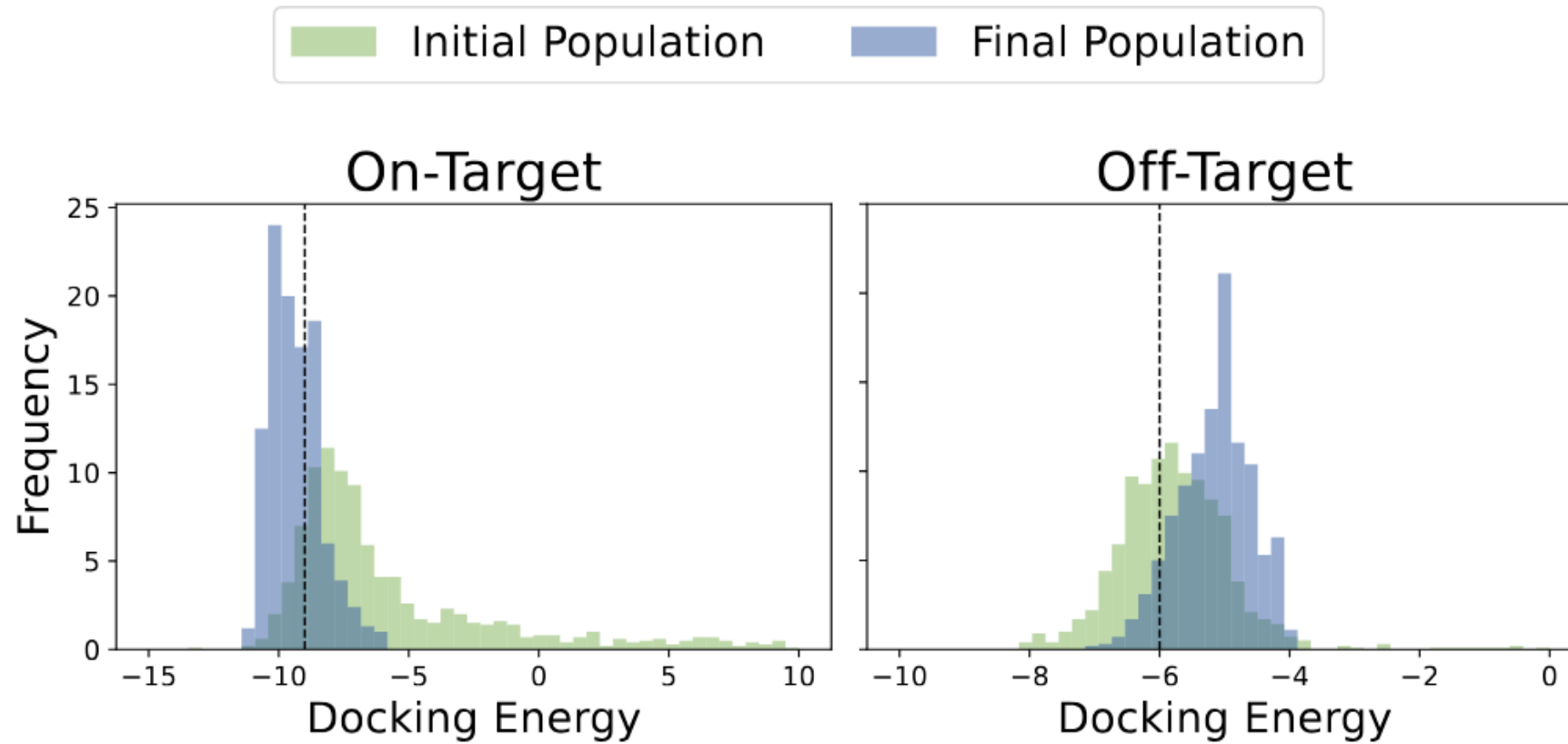
MUTATED MOLECULE  
CCOc1c(P(=O)(O)O)ccc(O)c(C(=O)O)c1

# Evaluating the fitness (without MD)

- We could, in principle, use **molecular dynamics** to simulate the binding between a putative solution and the ligand, but **it would take forever**
- We use **molecular docking** instead
  - Specifically, **AutoDock Vina 2.1**
  - GPU version to **accelerate the calculations** (single receptor / multiple ligand)
  - It returns a **numerical score** that summarizes the «strength» of the bond
- The synthesizability is assessed with the **Synthetic Accessibility** score (10=hard, 1=easy) calculated by RDkit

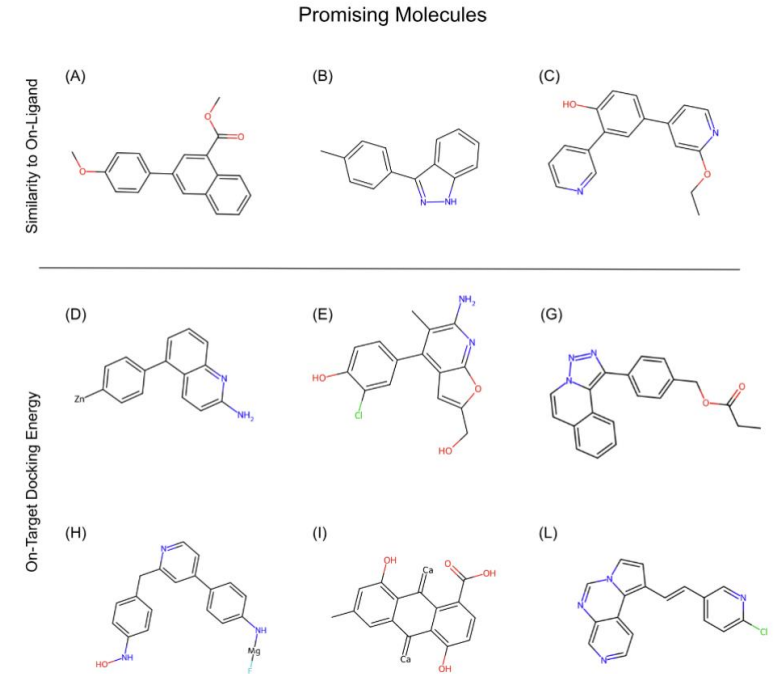


# Some results: on- and off-target energies



# Some results: a few new drugs

- We studied the **Beta1 receptor**, which is often targeted for **cardiac therapies**
  - Drugs targeting such receptors allow the **reduction of the heart rate** and blood pressure
  - However, drugs are often captured on the membrane and expelled by the **P-glycoprotein**, reducing the effectiveness of treatments
- We have the on-target (Beta1) and the off-target (P-glycoprotein) ligands
- In the figure you can see some **optimal drugs**

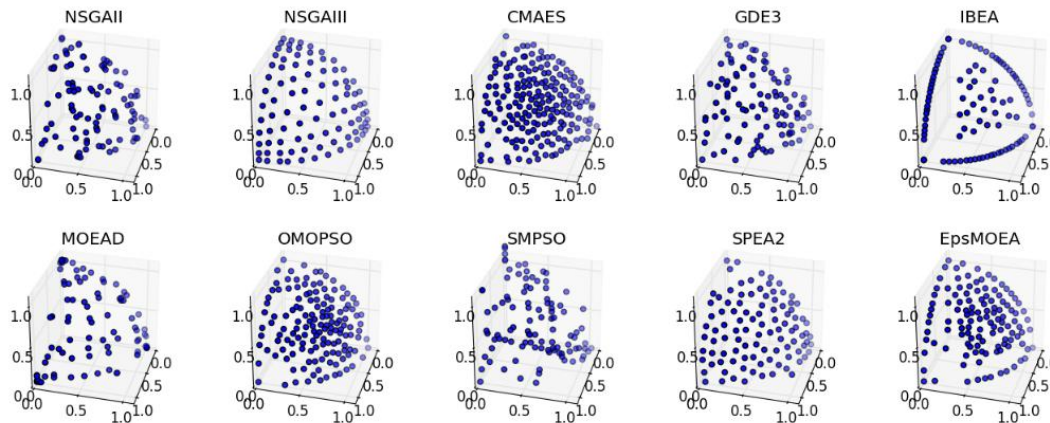


# Out of the box MOO: Platypus

---

## Platypus: MOO in python

- Supports a lot of widespread algorithms including OMOPSO, NSGA-II, NSGA-III, etc.
- Integrates several **MOO benchmark** functions (you can test your own algorithm)
- PRO: Easy to use, CON: Documentation
- Supports **automated testing** and visual comparison of performances (see Experimenter class)
- Supports **parallel computation** (using multi-processing, process pools, etc.)
- <https://github.com/Project-Platypus/>



# Multi-objective ACO

---



ACO has been extended to support multiple objectives

- E.g., BicriterionAnt [Iredi *et al.*, EMO 2001], Pareto ACO (PACO) [Doerner *et al.*, Ann Operat Res 2004]
- These algorithm share the idea of using **multiple pheromone matrices** (and, possibly, multiple heuristic matrices): **one matrix for each objective**
- Usually, only **Pareto-dominant ants can update** the pheromone matrices using **individual** (=specific matrix) or **global** (=many or all matrices) updates
- Pareto dominant ants are added to an **external archive**, which is **returned as solution**

**Multiple competing populations**, each with separate independent pheromone and heuristic matrices, can also be used

- See, e.g., COMPETants [Doerner *et al.*, 2003]
- Each colony tends to converge towards its **own objective**
- COMPETants uses “**spy**” **ants** to recombine information between colonies and mitigate the convergence to the extremes of the Pareto front

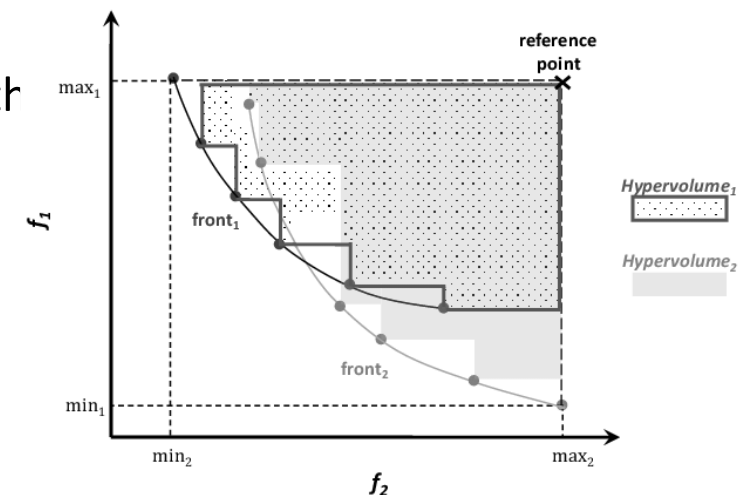
# Analyzing the performances of MOO algorithms

How can we **compare the performances** of two (or more) MOO algorithms?

- Is there a “quality measure” for a Pareto front?
- If we knew how the Pareto front is shaped we could calculate some “distance” from it, but that information is clearly unknown in real world scenarios
- **Several metrics exist** (e.g., Generational Distance, Spacing Metric, Extent of the Approximation set) but they can give **misleading information** about the quality of the Pareto front (see [\[Knowles \*et al.\*, 2006\]](#))

The most widespread and well accepted metrics is the **hypervolume**

- Measurement of the volume encapsulated by a **reference point** and the
- The larger the hypervolume, the better the performance
- See [\[Zitler \*et al.\*, Proc EMO 2007\]](#)



# Summary

---

- Optimization problems can be very difficult but global optimization meta-heuristics can provide effective solutions
- Sometimes, the objectives to be optimized are more than one: we can leverage multi-objective versions of the aforementioned algorithms (e.g., NSGA, OMOPSO)
- MOO implies Pareto domination: we are looking for the best approximation of the set of dominating solutions
- We can compare the performance of two algorithms using hypervolumes